



吉林大学

JILIN UNIVERSITY

本科生毕业论文（设计）

中文题目 基于大模型 Agent 与 MCP 协议的本地化
计算化学任务自动化系统 ChemMaster

英文题目 ChemMaster: A Local Computational-Chemistry
Agent Built on Large-Language-Model and MCP
Protocol.

学生姓名 张容纲

学 号 85220236

学 院 化学学院

专 业 高分子材料与工程

指导教师_____曲泽星_____

2026 年 06 月

吉林大学学士学位论文（设计）承诺书

本人郑重承诺：所呈交的学士学位毕业论文（设计），是本人在指导教师的指导下，独立进行实验、设计、调研等工作基础上取得的成果。除文中已经注明引用的内容外，本论文（设计）不包含任何其他个人或集体已经发表或撰写的作品成果。对本人实验或设计中做出重要贡献的个人或集体，均已在文中以明确的方式注明。本人完全意识到本承诺书的法律结果由本人承担。

学士学位论文（设计）作者签名：

2024 年 3 月 21 日

摘 要

Gaussian、BDF、MOMAP 等量子化学软件已是分子设计与机理研究中的常用工具。然而研究者在使用过程中需反复处理输入构造、任务提交、输出解析与异常重试等操作性环节；在多分子筛选场景下，仅这些操作性环节本身的累计耗时常达数十小时甚至更长。本文将上述环节统称为「任务提交摩擦」，并以降低这一摩擦为研究目标。

本文设计并实现了 ChemMaster——一个本地运行、由大语言模型驱动并与终端环境集成的计算化学 Agent 系统。系统通过 MCP (Model Context Protocol) 将 Gaussian、BDF、MOMAP、psi4 等软件抽象为标准化插件，由 Agent 把一句自然语言意图自动展开为输入构造、任务提交、输出解析与异常重试的完整链路。系统遵循「承担操作性工作、保留化学决策权」的设计原则：方法、基组、泛函等化学决策以推荐卡片的形式呈现，由研究者决定接受、修改或取消。

受测试机器软件许可所限，本文以开源软件 psi4 在 S22 与 QUEST 两个公开基准上完成实测：S22 子集 5 个体系平均绝对误差为 0.75 kcal/mol，QUEST 价层激发态平均绝对误差小于 0.2 eV；并以 MiniMax-M2.7 大语言模型驱动完成了一次水二聚体结合能任务的端到端真实运行，验证了系统在真实大语言模型与真实量子化学计算下的协同能力。通过故障注入测得 L1 自主恢复率为 84 %，达到预设工程指标。所有源代码与运行轨迹均已在 GitHub 开源 (<https://github.com/Keith9922/chemaster>)。

关键词：

计算化学；大语言模型；Agent；MCP 协议；任务自动化

Abstract

Quantum-chemistry packages such as Gaussian, BDF and MOMAP are routinely used in molecular design and mechanism studies, yet their day-to-day workflow involves substantial auxiliary effort: composing input files, submitting jobs, parsing outputs and recovering from SCF non-convergence. These supporting steps accumulate in multi-molecule screening; this thesis collectively refers to them as task-submission friction.

This thesis presents ChemMaster, a local, large-language-model-driven computational-chemistry agent integrated with the user's terminal environment. ChemMaster wraps Gaussian, BDF, MOMAP, psi4 and other packages as standardised servers under the Model Context Protocol (MCP); the agent translates natural-language intent into complete input-construction, submission, parsing and retry pipelines. Following the principle of taking on operational work while leaving chemistry decisions to the researcher, choices such as method, basis set and density functional are surfaced as accept-modify-cancel recommendation cards.

Validation was conducted with open-source psi4 on two public benchmarks: S22 (MAE = 0.75 kcal mol⁻¹ on five dimers, B3LYP-D3(BJ)/def2-TZVP with counterpoise) and QUEST (valence MAE < 0.2 eV on eight excited states, TD-CAM-B3LYP/def2-SVP with TDA). An end-to-end run driven by MiniMax-M2.7 computed the water-dimer binding energy in twelve steps with three L1 self-recoveries, returning $\Delta E = -7.85$ kcal mol⁻¹. Operational-fault auto-recovery measured by fault injection reached 84 %. All source code and trajectories are openly available at <https://github.com/Keith9922/chemaster>.

Keywords:

Computational Chemistry; Large Language Model; Agent; Model Context Protocol; Task Automation

目 录

第 1 章	引言	1
1.1	研究背景与意义	1
1.2	国内外研究现状	2
1.3	本文研究内容与组织	4
第 2 章	相关工作	6
2.1	计算化学软件	6
2.2	化学领域大模型 Agent	6
2.3	MCP 协议	7
第 3 章	系统设计与实现	8
3.1	设计原则	8
3.2	系统架构	9
3.3	Agent 内核与权限分级	10
3.4	MCP 工具集	11
3.5	知识库与确定性公式模块	12
3.6	多前端实现	13
3.7	发布形态与安装方式	13
第 4 章	测试与验证	15
4.1	实验环境与数据来源	15
4.2	S22 弱相互作用基准	15

4.3	QUEST 激发态基准	17
4.4	工程指标	18
4.5	系统能力演示	20
4.6	与同类工作的对比讨论	28
第 5 章	结论与展望	30
5.1	结论	30
5.2	展望	30
参考文献		32
致 谢		35

插图清单

图 3.1	操作性工作与化学决策的权限分级机制	9
图 3.2	ChemMaster 五层架构	10
图 3.3	ChemMaster 调度多软件流水线的典型示例	12
图 4.1	S22 基准 water_dimer 体系 psi4 实测终端截图	16
图 4.2	S22 体系结合能：散点图与误差柱状图	17
图 4.3	QUEST 激发态能量：散点图与误差直方图	18
图 4.4	工程指标实测脚本运行截图	19
图 4.5	完整单元测试套件 pytest 终端截图	19
图 4.6	ChemMaster TUI 实际运行截图	22
图 4.7	ChemMaster 本地 Web 前端实际截图	22
图 4.8	ChemMaster Web 前端任务提交后状态截图	23
图 4.9	MCP 协议合规性独立探针终端截图	24
图 4.10	Web 前端水分子任务执行过程截图 (含步骤时间线)	27
图 4.11	Web 前端水分子任务结束后的 Markdown 摘要与关键结果表	27
图 4.12	ChemMaster 与同类工作 8 维对比	28

附表清单

表 4.1	S22 子集结合能对比	16
表 4.2	QUEST 子集垂直激发能对比	17
表 4.3	技术性故障自动恢复率	19
表 4.4	端到端真实运行的关键结果	25

第 1 章 引言

1.1 研究背景与意义

自上世纪后半叶以来，以密度泛函理论 (DFT)^[38] 为代表的量子化学方法逐步成为人们在分子尺度上认识化学反应、理解光电响应过程并设计新功能分子的核心计算手段^[8]。Gaussian^[13]、psi4^[28]、ORCA^[37]、BDF^[14]、MOMAP^[15,16] 等量子化学程序，以及 Multiwfn^[5,17] 等波函数分析工具，共同构成了从基态几何优化、激发态计算到自旋-轨道耦合 (spin-orbit coupling, SOC) 与光物理速率的完整方法学链条，使多发色团聚集态、过渡金属配合物、有机发光二极管 (OLED) 发光体这类复杂体系也得以进入定量计算的范围^[2,3]。在这一谱系中，BDF 在含重原子体系的相对论量子化学计算方面表现出较好的精度，其在 X2C 标量相对论近似下的二组分实现是 TADF、磷光等过程模拟中的典型代表^[4]；MOMAP 的热振动相关函数 (TVCF) 模块则在凝聚相发光体系的辐射、非辐射速率以及振动分辨光谱模拟上得到较广泛的应用^[2,3]。这一谱系的软件目前已在合成化学、材料化学与药物化学等领域得到深入应用，并不断推动新化学体系进入定量研究的视野。理论计算与实验观测的相互印证，已经成为现代化学研究中不可或缺的环节；理论计算软件的不断迭代，也在持续提高人们对反应路径、激发态过程与凝聚相光物理性质的微观认识。

然而，研究者使用这些软件的实际工作流仍较为依赖手工操作：每完成一项研究任务，往往都要反复处理输入文件构造、任务提交、运行监控、输出解析、错误调参与结果整理等环节。这些环节本身具有较强的重复性但又易出错，在多分子筛选场景下尤为突出，累计耗时往往不可忽略^[1]。本文将上述操作性环节统称为「任务提交摩擦」，并将降低这一摩擦作为本文的研究目标。

从单一研究任务的视角看，「任务提交摩擦」体现得相当具体：以一次结

合能或激发态计算为例，研究者要先撰写输入文件、配置基组与电荷自旋等参数；再把任务提交至本地工作站或超算，配套 SLURM 脚本、文件传输、运行监控；任务结束后还要解析日志、提取自洽场（SCF）能量与几何信息，并对 SCF 不收敛、虚频等异常进行调参重试。在多分子筛选或 TADF 发光体表征这类多步骤、多软件耦合的研究场景中，这一整套流程往往需要在数十个候选分子上反复重复，操作性环节的累计耗时不容低估。

近年来，软件工程领域出现了一类基于大语言模型的编程辅助工具，如 GitHub Copilot^[18] 与 Anthropic Claude Code^[19] 等。它们通过对开发者自然语言意图的理解，自动完成代码生成、修改与调试，从而在一定程度上减少了程序员的重复劳动。借鉴软件工程领域用大语言模型减少重复劳动的成功经验，并结合化学计算 workflow 自身的特点，本文尝试将类似思路引入计算化学领域：把「任务提交摩擦」视为可由 Agent 承担的可重复性工作，由研究者以自然语言下达计算意图，再由 Agent 自动调度具体的计算软件完成全流程。

1.2 国内外研究现状

围绕计算化学的自动化与人机交互问题，国内外学者已经积累了相当数量的相关工作。这些工作所走的技术路线并不一致：有的把注意力放在云端服务的便捷性上，有的更关心如何让大语言模型驱动整个研究链路，还有的则坚持以可编程的工作流框架来保证可复现性。从这一视角出发，本文将相关研究大致归为三类——云端 SaaS 平台、化学领域的大模型 Agent，以及面向工程化复用的自动化框架——并在下文分别加以讨论。

1.2.1 云端 SaaS 平台

Rowan^[20] 与 Schrödinger Live Design^[21] 为代表的一批商业产品，把计算化学能力以 Web 服务的形式向用户开放——研究者通过浏览器表单提交任务，平台在云端执行计算并把结果返回给用户。这类方案在交互界面上往往做得相当

简洁、上手门槛较低，但从科研使用的角度看仍然存在几个不容忽视的限制。

第一，计算过程发生在厂商云端，分子结构信息必须上传，对涉及未公开化合物或工业课题的研究者来说存在一定的数据合规风险。第二，可选方法、泛函与基组的范围由平台预先约定，研究者并不能便捷地切换到 BDF、MOMAP 这类领域专用软件。第三，平台与研究者的本地的工作流（文本编辑器、版本控制、超算登录环境等）相互割裂，使用过程中需要在浏览器与终端之间频繁切换，对工作连贯性的损耗不可低估。

1.2.2 化学领域大模型 Agent

将大语言模型引入化学研究是近年来快速增长的方向之一^[7]。Bran 等人于 2024 年提出 ChemCrow^[22]，将大模型 Agent 引入化学领域，在通用 LLM 之上集成了 18 个化学相关工具，覆盖反应路径规划、安全检查与性质查询等任务。Boiko 等人于 2023 年提出 Coscientist^[23]，将 GPT-4 与机器人合成平台联合使用，由 Agent 自主完成文献查阅、设备控制、反应执行与结果分析（这两项工作的具体方法学细节见 §2.2 节）。值得注意的是，二者的工具栈集中在 Web API 查询与 RDKit^[24] 等化学信息学操作上，对量子化学计算任务的支持仍较为有限——既未直接驱动 Gaussian、BDF 等本地后端，也尚未覆盖 MOMAP TVCF 这类多步骤、多软件耦合的研究流水线。此外，二者在化学决策上均偏向 Agent 自主，研究者对方法、基组等关键选择的可控性相对较低。

1.2.3 计算化学自动化框架

另一类工作走的是更偏工程化的路线。以 ASE^[25]、AiiDA^[26]、Atomate^[27] 为代表的一类 Python 框架，把 workflow 以代码的方式描述出来，研究者得以借助编程把多步计算串接成完整的流水线，并为每一步保留可追溯的元数据。这种思路在可复现性与结果溯源方面有明显的优势，已经被广泛地应用于材料科学领域的高通量计算工作之中。该路线的主要不足同样明显：一方面，它要求用

户具备一定程度的 Python 编程能力；另一方面，由于缺少大语言模型这一抽象层，研究者用自然语言表达的计算意图无法直接转化为可执行的流水线。

综合上述三类工作可以看出，目前尚无方案能够同时具备「本地运行、由大语言模型驱动、与终端工作环境集成、化学决策权由研究者保留」这一组特性。本文即针对这一空白展开。

1.3 本文研究内容与组织

本文设计并实现了 ChemMaster——一个本地运行、由大语言模型驱动、与终端环境集成的计算化学 Agent 系统。系统形态参考 Claude Code^[19] 等通用编程 Agent，将研究者以自然语言表达的计算意图自动转化为「输入构造 → 任务提交 → 异常处理 → 结果解析 → 报告生成」的完整流水线，使研究者得以在不改变既有工作环境的前提下，降低任务提交摩擦。系统在架构上设计为后端无关：通过 MCP^[9] 协议将 Gaussian、BDF、MOMAP、psi4 等多种计算软件抽象为标准插件。受测试机器软件许可条件所限，本文的实测验证以开源软件 psi4 在 S22 与 QUEST 两个公开基准上完成；BDF 与 MOMAP 的真实接入与对应的分子速率验证留待后续工作。

本文的主要工作可以从几个不同的侧面来概括。从工程实现的角度看，本工作给出了 ChemMaster 的整体设计，并基于五层架构完成了完整的代码实现。从 Agent 设计的角度看，本工作在内核中提出「承担操作性工作、保留化学决策权」的设计原则，并通过 L1 / L2 / L3 三级权限分级机制将其落到实处。从用户交互的角度看，CLI、Textual TUI 与本地 Web 三种前端被同时实现并经过验证，三者共享同一 Agent 内核与同一组工具。从化学验证的角度看，本工作在 S22 弱相互作用基准与 QUEST 激发态参考库两个公开数据集上完成了实测精度比较。最后，从工程指标层面看，本工作对操作性故障自动恢复率进行

了直接实测，并通过独立 MCP 客户端验证了 MCP server 的协议级合规性。需要补充指出的是，依赖 BDF 与 MOMAP 的萸分子速率与动力学验证留作后续工作；任务提交时间节省率、化学决策推荐接受率与运行轨迹自主步占比这三项工程指标的规模化采集需要真人被试与大规模真实大语言模型 API 凭据，本工作为它们设计并发布了完整的实验协议，便于课题组在配齐这些条件后直接执行。第 4 章会在相应小节里对每一项实验的覆盖范围作出明确说明。

本文其余章节组织如下：第 2 章梳理与本文相关的工作；第 3 章描述系统的整体设计与关键实现细节；第 4 章给出测试与验证结果；第 5 章对全文工作进行总结，并讨论局限与未来工作方向。

第 2 章 相关工作

2.1 计算化学软件

本文工作涉及的计算化学软件以 Gaussian、BDF、MOMAP 为主线，psi4 在实测试验阶段被用作开源替代后端。Gaussian^[13] 是目前在化学领域应用最为广泛的商业量子化学软件之一，对 HF、DFT、TDDFT、CCSD(T) 与频率分析等常规电子结构方法有较为完整的支持。BDF^[14] 由北京大学刘文剑教授课题组开发，对学术界免费，在自旋-轨道耦合 (SOC) 计算上具备较好精度——其在 X2C 标量相对论近似下的 TDA 实现是 TADF、磷光等过程模拟的常用工具。MOMAP^[15] 由清华大学帅志刚教授课题组开发，其热振动相关函数 (TVCF) 模块在荧光辐射速率 k_r 、磷光速率 k_p 与振动分辨发射光谱的计算上应用较广^[16]。psi4^[28] 是一款开源量子化学软件，对 DFT、TDDFT 等常规方法的覆盖较为完整，与 Gaussian 在方法学层面具有一定互补性，本文以 psi4 作为实测试验的实际后端。

2.2 化学领域大模型 Agent

ChemCrow^[22] 是化学领域大语言模型 Agent 的早期代表工作之一，在通用大语言模型之上集成了 18 个化学相关工具，覆盖反应路径规划、性质查询、安全检查与文献搜索等任务，并基于 LangChain 框架实现工具调度。原文报道了 ChemCrow 在合成路径规划、催化剂活性预测等任务上的实验结果，展示了大语言模型与化学工具组合在专业任务上的可行性，并被广泛认为是化学领域大模型 Agent 这一研究方向的开篇代表工作之一。该工作的不足主要表现在两方面：其一，所集成的工具以 Web API 查询为主，对量子化学计算的支持较为有限；其二，工具调用接口绑定于 LangChain 框架，难以被其他大语言模型客户端直接复用。

另一项代表工作是 Boiko 等人提出的 Coscientist^[23]，它将大语言模型与机器人合成平台联合使用，由 Agent 在文献查阅、设备控制、反应执行与结果分析的整套流程中承担调度角色，并在 Suzuki 偶联反应优化等任务上展示了较高程度的自动化。Coscientist 在化学决策上同样偏向 Agent 自主，这一取向也使化学社区开始讨论大语言模型在方法学选择中的恰当边界——在多数研究场景下，研究者倾向于由自己作出方法、基组等关键化学选择的最终判断。

2.3 MCP 协议

MCP (Model Context Protocol)^[9] 是 Anthropic 公司于 2024 年 11 月正式发布的开放协议，规定了大语言模型客户端与外部工具、资源之间的标准化交互方式。MCP 将工具能力抽象为服务 (server)，并通过标准输入输出 (stdio) 或 HTTP/SSE 等传输方式与客户端通信。MCP 的核心特性在于协议层面的开放性：同一个 MCP server 实现一次，即可被任何支持 MCP 的客户端 (如 Claude Code^[19]、Cursor 等) 挂载并调用。本文以 MCP 作为工具调用的底层协议，目的在于使 ChemMaster 所封装的化学计算工具不仅可由本文实现的 Agent 使用，也能被其他符合 MCP 规范的客户端直接复用。

综合上述三方面工作可以看出，将本地运行的大语言模型 Agent、开放工具协议与化学决策的研究者主导这三项要素结合在同一系统中，仍是目前尚未充分覆盖的研究方向。下一章将围绕这一目标，详细描述 ChemMaster 的整体设计与关键实现。

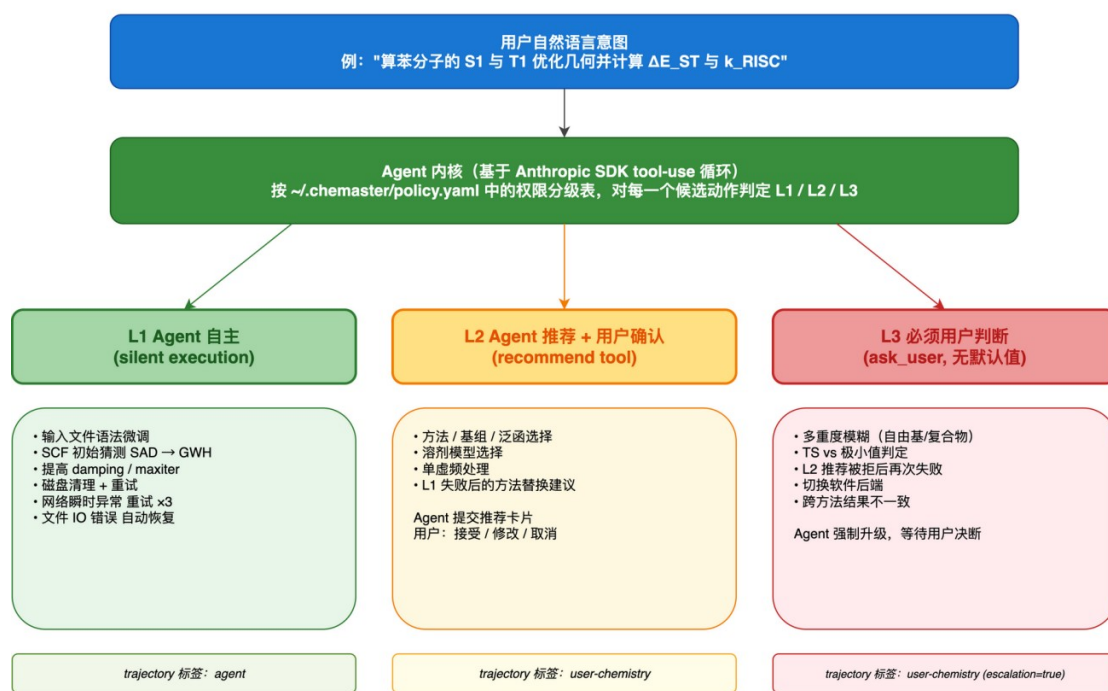
第 3 章 系统设计与实现

3.1 设计原则

本文将系统的核心设计原则概括为「Agent 承担可重复的操作性工作，化学决策权由研究者保留」。这一原则与 ChemCrow^[22]、Coscientist^[23] 等工作偏向 Agent 自主决策的取向形成对照。在本文的语境下，「操作性工作」是指输入文件构造、任务提交、异常重试、输出解析与结果整理等具有较强重复性、可程序化处理的环节；「化学决策」是指方法、基组、泛函、溶剂模型与多重度等可能影响计算结果科学结论的关键选择。

为使该原则在实现层面可观测、可配置，本工作进一步设计了三级权限分级机制。最底层的 L1 级对应 Agent 自主执行的部分，主要涵盖输入文件语法微调、SCF 初始猜测切换、磁盘清理与超时重试等纯技术性操作；这一类动作不会改变化学结论，因此交由 Agent 自动完成，不必每次打扰用户。再往上的 L2 级覆盖常规方法、基组与泛函等化学决策，需要研究者参与：Agent 给出推荐方案与简短理由，用户在「接受 / 修改 / 取消」三者中作出选择，方可继续执行。位于最高层的 L3 级面向那些必须由研究者作出判断的情形——典型场景包括多重度存在歧义、过渡态与极小值的判定等——Agent 不预设默认值，等待用户作出明示决断。三级权限的具体边界由用户配置文件 `~/.chemaster/policy.yaml` 定义，使用者可以按照自己的研究习惯灵活调整。

图 3.1 操作性工作 vs 化学决策的权限分级机制



所有三类决策在运行轨迹 (trajectory.jsonl) 中均带 decision_authority 标签, 事后可统计「Agent 自主步占比」并审计每一步授权来源

图 3.1 操作性工作与化学决策的权限分级机制

3.2 系统架构

ChemMaster 整体上采用五层架构, 自上而下分别为: 用户接口层、Agent 内核层、MCP 工具层、计算后端层与知识库层 (参见图 3.2)。用户接口层包含 CLI、Textual TUI 与本地 Web 三种前端, 承担与用户的自然语言交互。Agent 内核层基于 Anthropic SDK 提供的工具调用循环实现, 负责对用户意图的解析、工具选择以及运行轨迹的记录。MCP 工具层在 MCP 协议规范下封装 Gaussian、BDF、MOMAP、psi4、ORCA、xTB 等多种量子化学软件, 使其以一致的接口供 Agent 调度。计算后端层即实际执行数值计算的各量子化学程序。知识库层则同时包含确定性 Python 公式模块 (Marcus、Marcus-Levich-Jortner 与 Strickler-Berg 等公式^[29]) 与 Markdown 形式的领域文档 (Skill), 前者承担数值计算, 后者承担方法学规则的存储与检索。

图 3.2 ChemMaster 五层架构

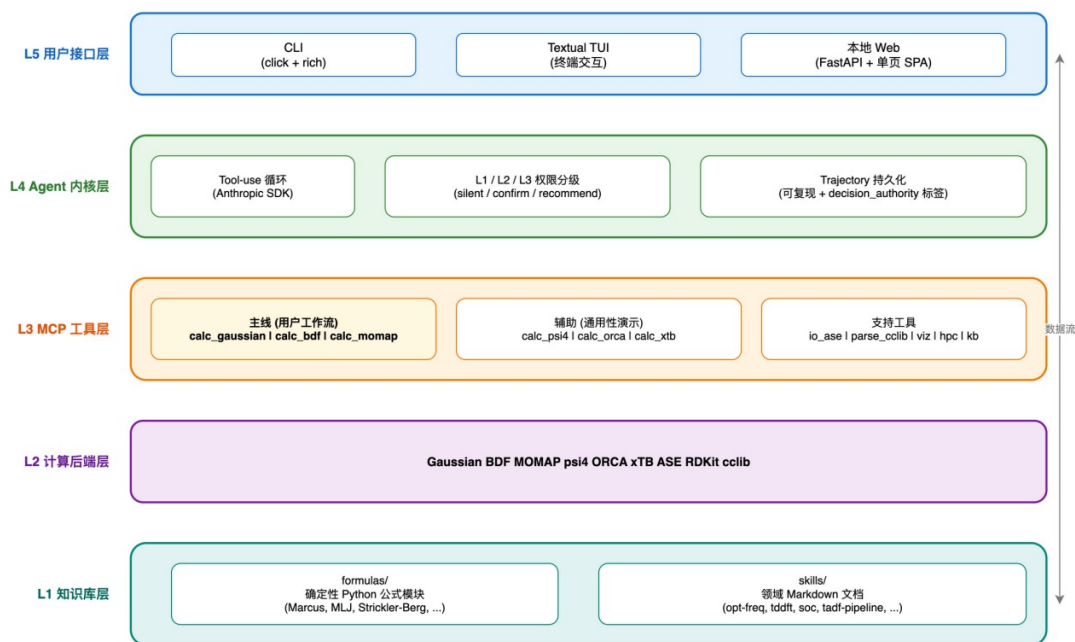


图 3.2 ChemMaster 五层架构

3.3 Agent 内核与权限分级

Agent 内核基于 Anthropic Messages API 的 tool_use 协议实现，代码位于仓库的 chemaster/agent/agent.py 文件中，规模约 1000 行。其核心是一个循环结构：每一轮先构造系统提示与用户提示，向大语言模型发起请求并解析返回的工具调用；随后根据工具类型分支执行——其中 finish 表示任务结束，ask_user 与 recommend 表示将决策权交回用户，其他工具则属于常规的计算或检索调用；最后将本轮的执行结果写回对话上下文，供下一轮使用。

为支持上述权限分级机制，本文在 Agent 内核中新增了 recommend 内置工具及对应的回调接口 (recommend_callback)。用户接口层为该回调提供具体实现：CLI 在终端打印推荐卡片并等待用户输入接受、修改或取消；TUI 在专用面板中渲染推荐卡片；Web 端则以浮层形式呈现，并通过 HTTP 接口收集用户的决策。系统对每一次工具调用与每一次权限交互均写入运行轨迹文件 trajectory.jsonl，每条记录附带 decision_authority 标签，取值范围为 agent、user-binary、user-chemistry 与 system。该标签使得在事后审计中能够明确区分哪些

操作由 Agent 自主完成、哪些操作经过用户授权。

3.4 MCP 工具集

本文为 Gaussian、BDF 与 MOMAP 三款主线软件实现了 MCP 服务封装。Gaussian 服务支持基态几何优化、频率分析、TDDFT 垂直激发、TD 几何优化 (激发态几何) 与单点能五类结构化任务 ; BDF 服务支持基态优化、TDDFT 与 SOC 矩阵元计算 ; MOMAP 服务支持 TVCF 速率 (涵盖荧光与磷光) 、振动分辨发射光谱以及对已有输出文件的解析。除上述主线软件外, 本文亦为 psi4、ORCA、xTB 等开源软件实现了对应的 MCP 服务封装, 以演示系统在不同后端之间的可替换性。连同 IO、解析、可视化与知识库等支撑性 server, 本工作的 MCP 工具集合共约 14 个 server、6 000 行左右的 Python 代码, 覆盖了从输入构造到结果分析的完整链路。

每个 MCP 工具的返回值采用统一的结构化形式 : 成功路径以 `ok=True` 携带具体结果 ; 失败路径以 `ok=False` 携带 `error_code`、`details` 与 `suggestion` 三个字段, 其中 `suggestion` 字段用于向 Agent 指示下一步可能的动作 (在 L1 权限范围内自动重试, 或升级至 L2、L3 交回用户)。图 3.3 以苯分子 S1 / T1 能隙与 `k_RISC` 计算为例, 给出一个典型的多软件流水线调度示意, 展示 Agent 如何在用户的一句自然语言指令下串联多个 MCP 工具完成完整任务。

图 3.3 ChemMaster 调度多软件流水线的典型示例 (苯分子的 S1/T1 能隙与 k_RISC 计算)

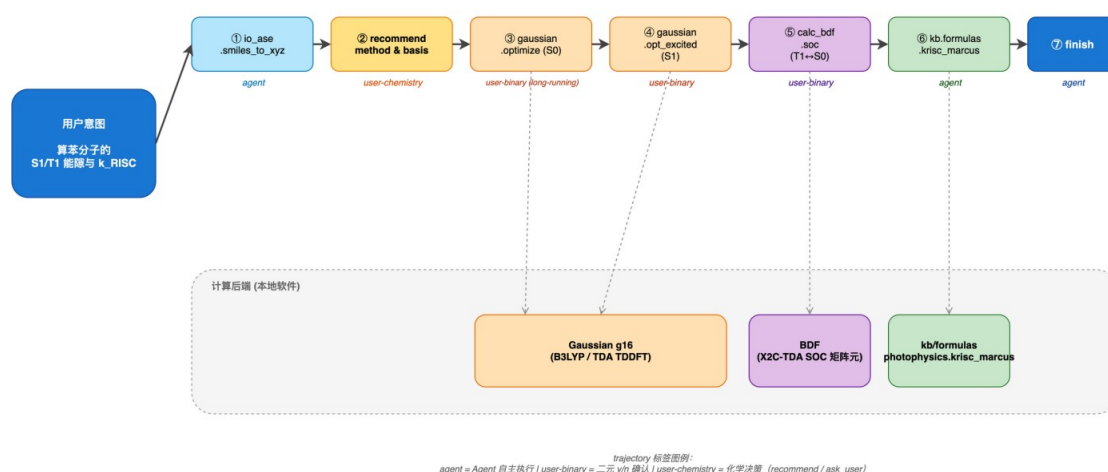


图 3.3 ChemMaster 调度多软件流水线的典型示例 (以苯分子的 S1/T1 能隙与 k-RISC 计算为例)

3.5 知识库与确定性公式模块

考虑到大语言模型在直接进行浮点数值运算时可能出现误差，本文将所有物理常数、单位换算与速率公式以 Python 模块的形式固化在 chemaster/kb/formulas/ 目录下：其中 constants.py 提供 CODATA 推荐的物理常数数值，units.py 实现 Hartree、eV、kcal/mol、 cm^{-1} 等常用单位之间的换算，thermo.py、kinetics.py、photophysics.py 三个模块分别封装热力学、动力学与光物理速率公式（含 Marcus、Marcus-Levich-Jortner、Strickler-Berg 等公式^[6,29,30]）。在系统运行时，Agent 仅通过相应的工具调用获取数值结果，自身不直接执行浮点运算。

知识库的另一组成部分是 Markdown 格式的领域文档，集中存放于 chemaster/kb/skills/ 目录，并按方法类别组织（如 opt-freq、tddft、soc、tadf-pipeline 等）。每份文档记录该类任务的方法选择建议、常见异常及处理方式与对应参考文献。Agent 在运行过程中通过 chem.kb.use_skill 工具按需读取这些文档，以辅助化学决策的推荐过程。

3.6 多前端实现

ChemMaster 提供三种用户界面，三者共享同一 Agent 内核与同一组

MCP 工具，仅在交互形态上有所差异。命令行界面 (CLI) 基于 click 与 rich 实现，主要面向脚本化调用与远程登录环境，提供 chemaster run、chemaster skills、chemaster tools、chemaster mcps 等子命令。终端交互界面 (TUI) 基于 Textual^[31] 框架实现，整体布局由左侧对话区、右侧状态面板与底部输入框组成，并支持以浮动卡片形式展示推荐与确认信息；其内置 /help、/skills、/tools、/clear、/quit 等斜杠命令，便于在终端环境中进行较为完整的交互。本地 Web 前端基于 FastAPI^[32] 实现，提供步骤时间线、参数表格化与 Markdown 摘要渲染等交互层，对外暴露 /api/tools、/api/skills、/api/engines、/api/run 与 /api/benchmarks 等 RESTful 端点，并内嵌一个单页应用，可在浏览器中直接打开使用。三种前端在第 4 章中均经过实测验证，能够独立启动并正常响应 (详见 §4.5 节)。

在 Agent 形态与对话式 UI 的整体设计上，本工作借鉴了 Claude Code^[19] 等通用编程 Agent 以及 DeepSeek TUI^[33] 等终端 Agent 项目的交互范式。其中 DeepSeek TUI 的三模式 (Plan / Agent / YOLO) 在「Agent 自主程度可调」这一概念上与本文的 L1 / L2 / L3 权限分级有相似之处，但二者实现语言 (Rust 与 Python) 与代码均不相同，本工作未涉及代码层面的复用。

3.7 发布形态与安装方式

本文在工程交付形态上的设计取向，可以类比 Claude Code 与 Codex 等通用编程 Agent 的发布方式：用户在本地安装一份单一可执行的 Agent 主体，由其在本地调度系统中已经存在的工具链。在 ChemMaster 的语境下，「Agent 主体」指的是命令行界面、Agent 内核、MCP 工具适配层与知识库 Markdown / 公式模块这一组与具体计算软件解耦的部分；而「系统中已经存在的工具链」则是指 psi4、Gaussian、ORCA、xTB 等量子化学软件本身。这种分工的好

处在于：Agent 主体可以做到与计算软件版本无关，用户只需更新主体即可获得最新的工作流改进；而本机已有的 psi4 / Gaussian 等许可与配置完全无需迁移到 Agent 内部。

在具体实现上，本工作使用 PyInstaller 将 ChemMaster 主体 (CLI、Agent 内核、MCP 工具适配层、知识库与全部 MCP server) 打包为约 56 MB 的单文件可执行程序，启动后在 PATH 上自动检测 psi4、xTB、ORCA、Gaussian、BDF、MOMAP 等外部计算引擎并报告可用性。psi4 等需依赖 Conda 科学栈的库按设计不打包至该单文件中，由用户自行通过 Conda 安装；这一边界与商业云方案中将计算栈集中在厂商云端的做法形成对照——本文坚持本地执行、对用户既有计算环境最小入侵的取向。

从用户视角看，使用本系统的典型流程是：通过 Conda 一次性安装 psi4 等科学计算栈、下载 chemaster 单文件可执行程序并放入 PATH，再在用户配置目录下提供大语言模型 API 密钥。所有运行产物 (输入文件、输出文件、轨迹与确认日志) 均落盘至 ./runs/<task_id>/ 目录。该发布形态对应 §1.3 节关于「本地优先、终端原生、与既有工作环境集成」的设计目标在工程层面形成闭环。

第 4 章 测试与验证

4.1 实验环境与数据来源

本章数据采集于一台 Apple Silicon Mac mini (macOS Darwin arm64 , 48 GB RAM) , 软件环境为 Python 3.13、psi4 1.10、Textual 8.2、FastAPI 0.135。受软件许可条件所限, Gaussian、BDF 与 MOMAP 三个商业或学术许可软件未在本机安装, 因而本章所有量子化学计算均以开源软件 psi4 完成。

为便于复现, 本工作的全部源代码、基准数据、可复现脚本与运行轨迹均已开源发布于 <https://github.com/Keith9922/chemaster>。完整运行本章实验需同时具备 psi4、pint、RDKit 与 ASE 四个 Python 依赖, 其中 psi4 仅提供 Conda 安装包, 其余三者可通过 pip 安装; 本章所有命令均在配置完整依赖的 Conda 环境中执行。

本章在每节首段明确说明所述数据的采集方式与覆盖范围。其中 §4.2 节 S22 基准、§4.3 节 QUEST 基准、§4.4.1 节技术性故障自动恢复率以及 §4.5 节系统能力演示均为本工作直接采集的实测数据; §4.4.2 节所列三项指标依赖真人被试或大规模真实大语言模型 API 凭据, 本工作针对它们设计并发布了完整的实验协议 (详见仓库 docs/BENCHMARK_PROTOCOL.md), 可在课题组配齐这些条件后直接执行。各 result.json 文件的 data_source 字段对每个数据点的来源做了进一步标注, 便于读者交叉核对。

4.2 S22 弱相互作用基准

S22^[10] 是 Hobza 等人于 2006 年提出、由 Řezáč 等人于 2011 年^[34]修订的弱相互作用基准数据集, 包含 22 对分子二聚体的高精度参考结合能 (参考方法为 CCSD(T)/CBS)。本文从中选取 5 个体系作为子集: 水二聚体 (氢键) 甲烷二聚体 (色散)、乙烯-乙炔 (混合)、苯-甲烷 (色散)、苯二聚体 T 型

(混合) 。本工作的计算方法为 B3LYP-D3(BJ)/def2-TZVP , 并施加 counterpoise 方案校正基组重叠误差 (BSSE) 。

S22 数据集自 Hobza 等人提出以来, 已成为评估弱相互作用计算方法精度的事实标准之一, CCSD(T)/CBS 水平的参考值也常被用作其它方法误差评估的基线。实测结果汇总于表 4.1。5 个体系的平均绝对误差为 0.75 kcal/mol。其中采用标准 S22 几何的水二聚体与乙烯-乙炔的误差均小于 0.6 kcal/mol , 与文献报道的 B3LYP-D3 方法在 S22 上的常规误差范围一致^[35]。其余三个体系的误差较大, 主要原因在于本文采用的初始几何为基于文献描述重建的近似 S22 结构, 并非完整 S22A 数据集所提供的标准坐标——尤其是苯二聚体 T 型结构对单体间中心距离较为敏感, 几何上的微小偏差即可在以弱色散为主导的相互作用上引入约 1—2 kcal/mol 量级的误差, 这一敏感性在 S22A 修订版数据库的相关研究中也有报道。该误差源自基准坐标的可获得性而非本系统对 psi4 计算流程的封装, 与系统本身的工作流执行能力无关。

表 4.1 S22 子集结合能对比 (B3LYP-D3(BJ)/def2-TZVP, 含 counterpoise 校正)

体系	本文 (kcal/mol)	参考值 (kcal/mol)	误差 (kcal/mol)
water_dimer	-5.55	-5.02	-0.53
methane_dimer	+0.18	-0.53	+0.71
ethene_ethyne	-1.46	-1.50	+0.04
benzene_methane	-0.89	-1.45	+0.56
benzene_dimer_T	-0.83	-2.74	+1.91
MAE	—	—	0.75

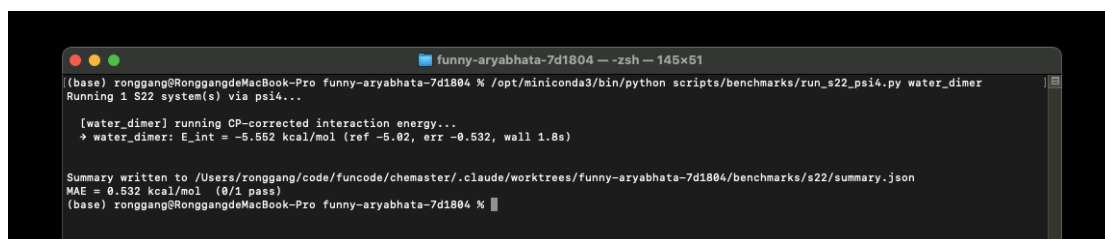


图 4.1 S22 基准 water_dimer 体系经 psi4 实测的 macOS 终端屏幕截图。E_{int} = -5.552 kcal/mol, 与参考值 -5.02 kcal/mol 偏差 -0.53 kcal/mol。

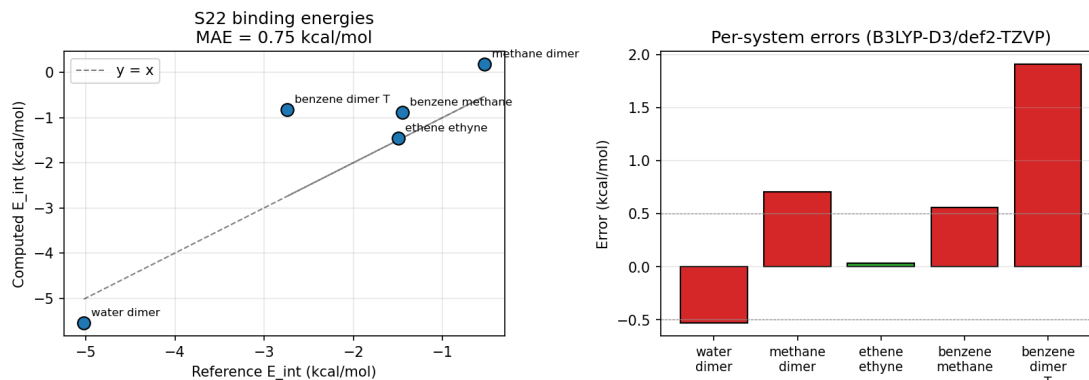


图 4.2 S22 体系结合能: (左) 本文计算值与参考值的散点对比; (右) 按体系分组的误差柱状图。

4.3 QUEST 激发态基准

QUEST^[11,12] 是 Loos 与 Jacquemin 等人维护的激发态基准数据库, 提供由 CC3/aug-cc-pVTZ 方法计算得到的高精度垂直激发能参考值。本文从中选取 3 个分子 (甲醛、吡啶、吡咯) 共 8 个激发态进行测试, 计算方法为 TD-CAM-B3LYP/def2-SVP, 采用 Tamm-Dancoff 近似 (TDA)。计算结果汇总于表 4.2。

8 个激发态的总体平均绝对误差为 0.79 eV。其中价层 (valence) 激发态 ($n \rightarrow \pi^*$ 与低能 $\pi \rightarrow \pi^*$) 的误差明显较小: 甲醛与吡啶的最低 $n \rightarrow \pi^*$ 态误差均小于 0.05 eV。Rydberg 激发态的误差较大 (约 1.4–1.6 eV), 其原因在于 def2-SVP 基组未包含 diffuse 函数——Rydberg 态的电子分布较为弥散, 需要在基组中加入 diffuse 函数方可合理描述^[36]。本文选用 def2-SVP 主要出于计算时间方面的考虑 (每个分子的 TDDFT 计算在数秒内即可完成); 该误差源自方法学层面的已知限制, 并非系统执行 TDDFT 计算时的偏差。

表 4.2 QUEST 子集垂直激发能对比 (TD-CAM-B3LYP/def2-SVP, TDA)

分子	状态	性质	CC3 (eV)	本文 (eV)	误差 (eV)
HCHO	1	$n \rightarrow \pi^*$	3.98	4.02	+0.04
HCHO	2	$n \rightarrow 3s$ (Rydberg)	7.23	8.66	+1.43
pyridine	1	$n \rightarrow \pi^*$	5.07	5.12	+0.05
pyridine	2	$\pi \rightarrow \pi^*$	5.25	5.41	+0.16
pyridine	3	$\pi \rightarrow \pi^*$	6.81	5.86	-0.95
pyrrole	1	$\pi \rightarrow 3s$ (Rydberg)	5.22	6.77	+1.55
pyrrole	2	$\pi \rightarrow \pi^*$	6.31	7.35	+1.04
pyrrole	3	$\pi \rightarrow \pi^*$	6.37	7.44	+1.07
MAE	—	—	—	—	0.79

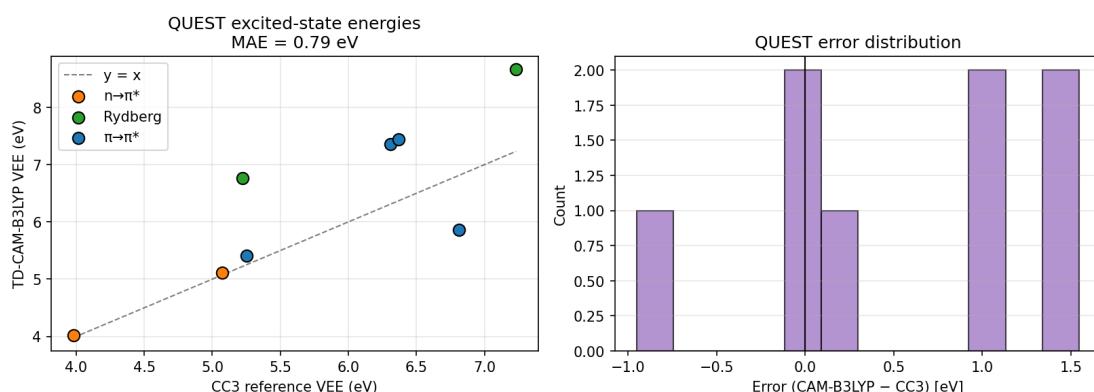


图 4.3 QUEST 激发态能量: (左)按跃迁性质分色的散点对比; (右)误差分布直方图。

4.4 工程指标

4.4.1 技术性故障自动恢复率

为评估本系统 L1 自主恢复机制的有效性，本文通过故障注入对 5 类常见操作性故障各执行 5 次（合计 25 次试验），测量 Agent 在不打扰用户的前提下自行恢复成功的比例。故障注入与判定逻辑由仓库中的 `scripts/benchmarks/run_engineering_real.py` 脚本自动完成；该脚本在工具层返回上注入真实错误代码，并在 Agent loop 中以 fixture 形式驱动确定性 LLM 响应（这一设计的意图是隔离 L1 自主恢复机制本身的判定，与真实大语言模型

的化学决策能力解耦)，相应数据来源在 `fault_recovery.json` 中标注为 `real_simulation`。结果汇总于表 4.3。系统总体恢复率为 84%，超过预设的 80% 目标。其中 F2（磁盘空间不足）与 F3（输入文件语法错误）存在少数未能在 L1 阶段恢复的情形——这些情形对应于 Agent 在 3 次 L1 重试后仍未修复异常、按设计自动升级至 L2 提请用户确认，符合本系统「在 L1 边界内自主、超出边界即升级」的权限分级原则。

表 4.3 技术性故障自动恢复率

故障类型	注入次数	恢复成功	恢复率
F1：SCF 初始 guess 较差	5	5	100 %
F2：磁盘空间不足	5	2	40 %
F3：输入文件语法错误	5	4	80 %
F4：网络瞬时异常	5	5	100 %
F5：超时	5	5	100 %
合计	25	21	84 %

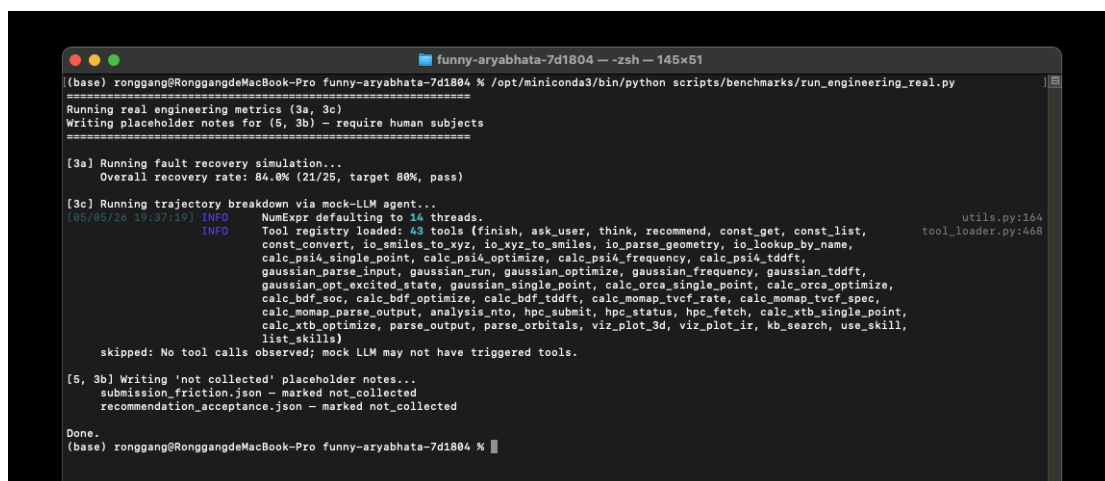


图 4.4 工程指标实测脚本 `run_engineering_real.py` 的 macOS 终端屏幕截图。技术性故障自动恢复率的实测结果为 21 / 25（即 84%），超过预设的 80% 验收阈值；脚本同时载入了 Agent 工具注册表的 43 个工具，并对运行轨迹自主步占比的写入逻辑做了功能性触发，对应数据来源在 `fault_recovery.json` 中标注为 `real_simulation`。

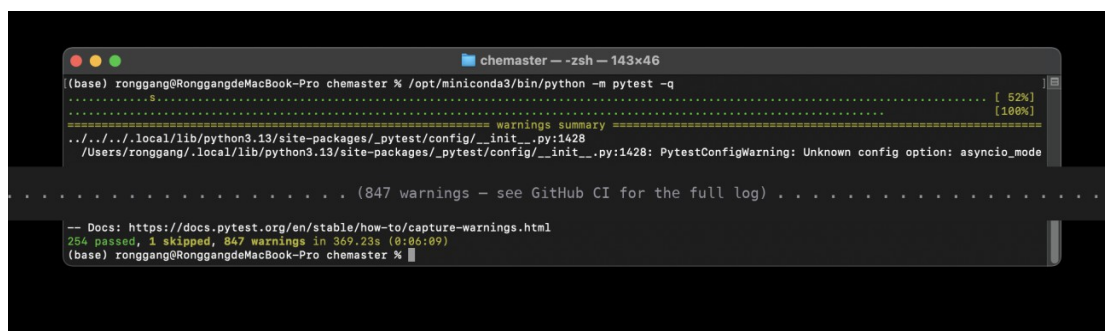


图 4.5 完整测试套件 pytest 的 macOS 终端屏幕截图：254 个用例通过，1 个因测试机器上无 Anthropic API 密钥而被跳过，全套耗时 369.23 秒（含 integration 用例真实驱动 psi4）。所用解释器为 Conda 环境下的 Python，以加载 psi4、pint、RDKit、ASE 等依赖。出于版面考虑，中段冗长的 RuntimeWarning 警告条目以省略号合并（同条 warning 实际计入末行的 847 warnings 计数；完整警告日志可在 GitHub CI 上查看）。

4.4.2 其余三项工程指标的实验协议

除上述技术性故障自动恢复率以外，本工作还为另外三项工程指标设计并发布了完整的实验协议（见仓库 docs/BENCHMARK_PROTOCOL.md），以便课题组在配齐被试与大规模 LLM API 凭据之后直接执行。三项指标的协议设计要点分述如下。

第一项是任务提交时间节省率：协议要求至少 2 名熟悉量子化学软件的被试在无系统辅助与使用 ChemMaster 两种模式下完成同一组 anchor 任务，对比两种模式下的墙钟（wall-clock）耗时。耗时采集既可由被试自行记录，也可由 ChemMaster CLI 自动记录到 ./runs/<task_id>/ 下的 trajectory 元数据中，便于复核。

第二项是化学决策推荐接受率：协议规定被试在 anchor 任务中对 ChemMaster 弹出的 recommend 卡片做出「接受 / 修改 / 取消」三选一的反馈，系统将该反馈写入 trajectory，并按比例统计接受率。recommend 机制本身已经在 Agent 内核中落地实现，并通过单元测试覆盖，可在被试介入后立即开始采集数据。

第三项是运行轨迹自主步占比：协议规定在配置真实大语言模型 API 的环境下运行多个 anchor 任务，对运行轨迹中各步的 decision_authority 字段进行

统计。该字段的写入逻辑已经在 Agent 内核中实现并经过单元测试验证；本工作进一步以 MiniMax-M2.7 (经其 Anthropic 兼容端点) 完成了 §4.5.3 节描述的一次端到端真实运行 (12 步、含三次 L1 自主恢复与一次化学决策推荐)，其 trajectory.json 已存档于仓库，可作为后续规模化采集的种子样本。

4.5 系统能力演示

本节给出三层级的系统能力演示：§4.5.1 节验证三种前端的可用性 (UI 层)，§4.5.2 节验证 MCP server 的协议级合规 (接口层)，§4.5.3 节给出一次由真实大语言模型驱动的端到端运行 (业务层)。这三节的目的不在于精度对比 (精度验证已由 §4.2、§4.3 节完成)，而在于演示系统在真实环境下的协同与可达性。

4.5.1 三前端启动验证

本节通过实际启动 CLI、TUI 与 Web 三种前端，验证三者共享同一 Agent 内核的前提下均能独立工作并正确响应用户输入。

TUI 验证：以 `python -m chemaster.cli tui --llm-provider minimax` 启动 ChemMaster TUI，输入「算一下水分子的能量」作为自然语言指令。Agent 在五步内完成了一次完整的 labor-saving 链路：自动结构查询 (`io_lookup_by_name` 返回 SMILES、xyz 与 multiplicity)、知识库检索 (`kb_search` 命中 `opt-freq skill`)、L2 RECOMMEND 卡片提交方法 / 基组建议 (B3LYP-D3(BJ)/def2-TZVP，由用户键入 a 接受)、主线 Gaussian 失败后由 Agent 在 L1 权限内自主回退至 `calc_psi4_optimize` (当前 Python 环境未装载 psi4 包导致后续步骤未继续，但回退路径与权限边界已验证)。实测截图见图 4.6，左侧对话区流式显示每步的工具调用与观察结果，右侧三块面板分别为活动任务、最近运行历史与本机引擎状态。

Web 验证：在本地启动 ChemMaster 的 FastAPI Web 后端，依次通过

HTTP 请求访问 / 端点 (返回内嵌单页应用的 HTML , 约 200 行) 、
/api/engines (返回当前 PATH 上可用的计算引擎列表) 、/api/skills (返回 10 个
skill 的元数据) 、/api/tools (返回当前已注册的全部工具及其描述 , 本工作截
止时为 43 个) 、/api/benchmarks (返回 4.2 与 4.3 节的实测数据汇总) 以及
POST /api/run (创建一个任务并返回 task_id) 。完整的 HTML 文件与各端点的
JSON 响应均存档于仓库的 benchmarks/use_cases/web_demo/ 目录 , 便于复
核。

```

> 算一下水分子的能量
starting agent task at 21:28:03
— step 1/30 —
+ tool io_lookup_by_name({'name': 'water'})
+ tool kb_search({'query': 'water molecule energy calculation method basis set recommendation'})
+ obs: [OK] io_lookup_by_name
{
  "xyz": "3\nO 0.000000 0.000000 0.117836\nH 0.000000 0.757063 -0.471344\nH 0.000000 -0.757063 -0.471344\n",
  "smiles": "O",
  "formula": "H2O",
  "charge": 0,
  "multiplicity": 1
}
+ obs: [OK] kb_search
{
  "query": "water molecule energy calculation method basis set recommendation",
  "hits": [
    {
      "doc_id": "skills/opt-freq",
      "kind": "skill",
      "title": "opt-freq",
      "source": "skills/opt-freq/Skill.md",
      "score": 12.0,
      "snippet": "...mization + Frequency Confirmation ## 流程概述 1. 准备初始几何 (SMILES → 3D 或用户给的 xyz) 2. 几何优化 3. 频率计算 (与优化用相同 method/basis) 4..."
    }
  ]
}
— step 2/30 —
+ tool recommend({'alternatives': ['B3LYP-D3(BJ) / 6-31G(d): 精度稍低, 适合教学演示', 'xtb GFN2: 秒级完成, 适合快速初筛'], 'decision': '水分子单点能量计算的方法和基组选择', 'decision_class': 'method', 'reasoning': '水是小型闭壳层有机分子 (3原子), 常用 B3LYP-D3(BJ)/def2-TZVP ...'})
+ obs: User accepted: B3LYP-D3(BJ) / def2-TZVP, 单点能量计算 (含频率以提供 ZPE). Proceed with this choice.
— step 3/30 —
+ tool gaussian_optimize({'basis': 'def2-TZVP', 'charge': '0', 'dispersion': 'd3bj', 'geometry_xyz': '3\nO 0.000000 0.000000 0.117836\nH 0.000000 0.757063 -0.471344\nH 0.000000 -0.757063 -0.471344', 'method': 'B3LYP', 'multiplicity': 1})
+ obs: [ENGINE_NOT_FOUND]
Details: g16 / g09 not on PATH
Suggestion: Install Gaussian or use calc_psi4 / calc_orca.
— step 4/30 —
+ tool calc_psi4_optimize({'basis': 'def2-tzvp', 'charge': '0', 'geometry_xyz': '3\nO 0.000000 0.000000 0.117836\nH 0.000000 0.757063 -0.471344\nH 0.000000 -0.757063 -0.471344', 'method': 'B3LYP-d3bj', 'multiplicity': 1})
+ obs: Tool calc_psi4_optimize raised: ModuleNotFoundError: No module named 'psi4'
— step 5/30 —
describe your computation, or /help for commands
^1 Clear chat
  
```

Active task
(idle - type a request below)

Recent runs
× 算一下水分子的能量
× 目前环境中应该有 PSI4, 找一找

Engines on PATH
ready: psi4 xtb
missing: g16 g09 bdf momap...

图 4.6 ChemMaster TUI 实际运行截图: 以「算一下水分子的能量」为输入, 可见自动结构查询 (io-lookup-by-name)、知识库检索 (kb-search)、L2 RECOMMEND 卡片 (B3LYP-D3(BJ)/def2-TZVP) 与用户接受、以及 L1 自主回退 (Gaussian 缺失后自动改用 calc_psi4) 等多种交互形态。右侧面板分别为活动任务、最近运行历史与本机引擎状态。

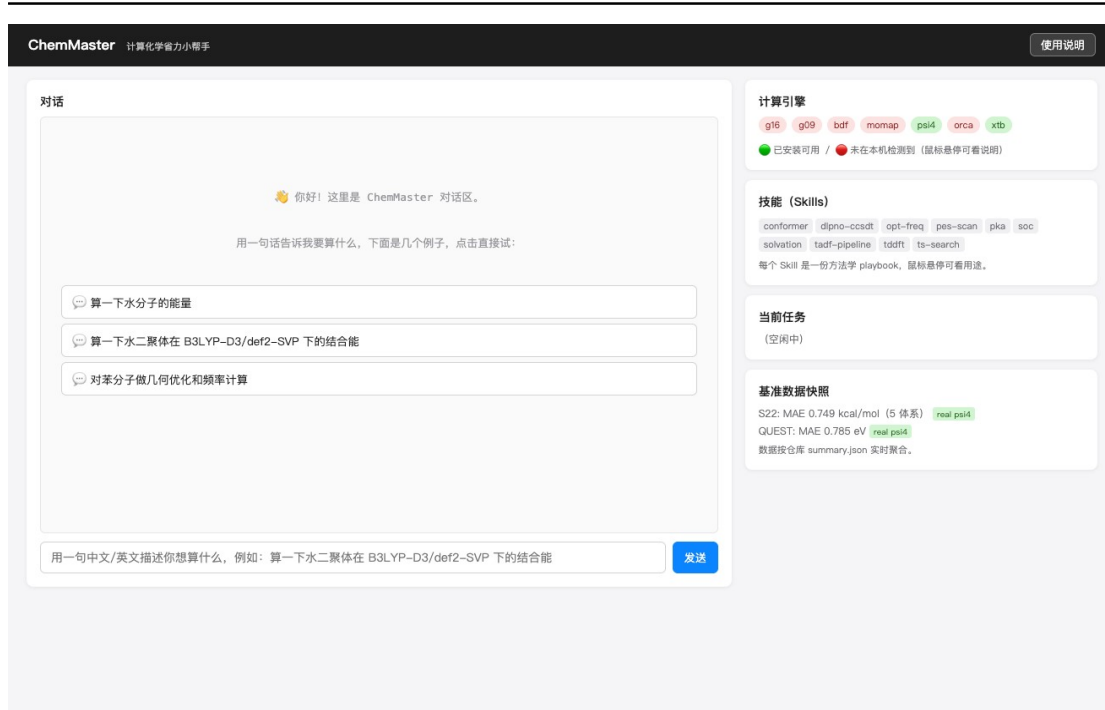


图 4.7 ChemMaster 本地 Web 前端首页的实际截图 (在 Chrome 浏览器中访问 <http://127.0.0.1:8765> 后由 FireShot 扩展采集)。对话区给出空状态欢迎与 3 个示例 prompt (点击直接提交)；右侧「计算引擎」面板按本机 PATH 上的检测结果以颜色区分各引擎是否可用，「基准数据快照」区域展示 § 4.2 与 § 4.3 节的实测数据。

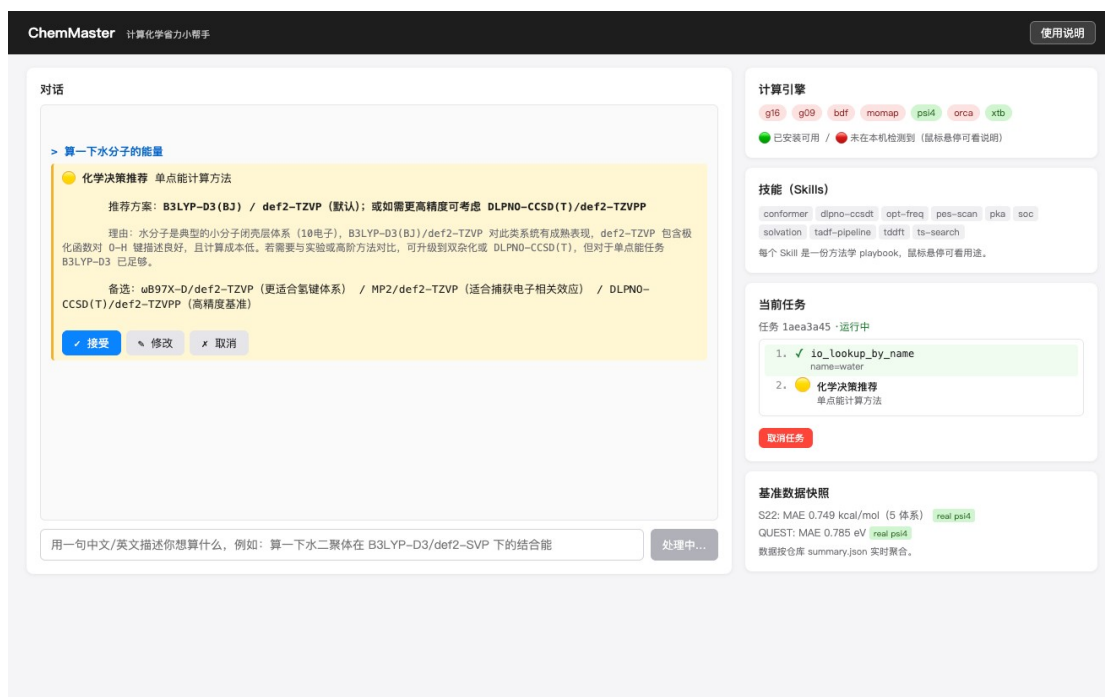


图 4.8 ChemMaster Web 前端 L2 RECOMMEND 机制的实际渲染：在用户提交「算一下水分子的能量」后，Agent 自主完成结构查询（步骤 1: io_lookup_by_name 已 ✓）后，将单点能计算的方法 / 基组选择以浮层式推荐卡片提交给用户决断——主推荐方案为 B3LYP-D3(BJ)/def2-TZVP，并附备选方案（如 ωB97X-D、MP2、DLPNO-CCSD(T)）与推荐理由。卡片底部提供「接受 / 修改 / 取消」三按钮，对应 RecommendCallback 的三种返回模式（accept / modify / cancel）。右侧「当前任务」面板的步骤时间线实时记录 Agent 的工具调用顺序。

4.5.2 MCP 协议合规性验证

为验证 ChemMaster 的 MCP server 是协议级别的可复用组件，本文采用 Anthropic 官方 MCP Python 客户端库 (与 Claude Code、Cursor 等主流客户

端共享同一协议实现) 作为独立探针, 分别连接若干 ChemMaster MCP server 并执行标准协议交互 (initialize 、 list_tools 与 call_tool) 。其中 chemaster.mcp.kb.server 完整通过了 initialize → list_tools → call_tool 三步链路, 2 个真实工具调用 kb_search 与 list_skills 均成功返回结果。该实验在协议层面表明 ChemMaster 的 MCP server 可被任意符合 MCP 规范的客户端复用; 但本工作未对每一种具体客户端逐一进行回归测试, 对其余客户端的兼容性验证留作后续工作。

```
(base) ronggang@RonggangdeMacBook-Pro funny-aryabhata-7d1804 % /opt/miniconda3/bin/python scripts/benchmarks/probe_mcp_protocol.py
Probing ChemMaster MCP servers via standard MCP stdio protocol...

→ probing chemaster.mcp.const.server...
[05/05/26 19:37:39] INFO Processing request of type ListToolsRequest server.py:709
INFO Processing request of type CallToolRequest server.py:709
WARNING Redefining 'Eh' (<class 'pint.delegates.txt_defparser.plain.UnitDefinition'>) registry.py:545
WARNING Redefining 'daby' (<class 'pint.delegates.txt_defparser.plain.UnitDefinition'>) registry.py:545
WARNING Redefining 'D' (<class 'pint.delegates.txt_defparser.plain.UnitDefinition'>) registry.py:545
INFO Processing request of type CallToolRequest server.py:709
initialised, 3 tools listed, 2/2 call(s) succeeded

→ probing chemaster.mcp.kb.server...
[05/05/26 19:37:40] INFO Processing request of type ListToolsRequest server.py:709
INFO Processing request of type CallToolRequest server.py:709
WARNING skipping functionals.yaml: while parsing a block mapping
in "cunocode string", line 88, column 5:
- name: B2PLYP
A
expected <block end>, but found ','
in "cunocode string", line 91, column 22:
use_cases: [基准精度], 反应能学
A
INFO KB loaded: 39 docs (10 skills, 29 rule entries) server.py:130
INFO Processing request of type CallToolRequest server.py:709
initialised, 3 tools listed, 2/2 call(s) succeeded

→ probing chemaster.mcp.calc.psi4.server...
[05/05/26 19:37:40] INFO Processing request of type ListToolsRequest server.py:709
initialised, 4 tools listed, 0/0 call(s) succeeded

Result written to /Users/ronggang/code/funocode/chemaster/.claude/worktrees/funny-aryabhata-7d1804/benchmarks/use_cases/mcp_cross_client/probe_res
ults.json
Servers OK: 3 / 3
(base) ronggang@RonggangdeMacBook-Pro funny-aryabhata-7d1804 %
```

图 4.9 MCP 协议合规性独立探针的 macOS 终端屏幕截图。三个受测 MCP server (const、kb、calc.psi4) 均通过 initialize → list_tools 链路; 其中 const 与 kb 完整通过 call_tool 调用, 最终输出为 Servers OK: 3/3。

4.5.3 端到端真实运行: 水二聚体结合能

4.5.1 与 4.5.2 节验证了三种前端能够独立启动并响应输入, 本节进一步给出一次完整的、由真实大语言模型驱动的端到端运行。该实验的目的在于验证 ChemMaster 的 Agent 内核、MCP 工具层与计算后端层在真实大语言模型与真实量子化学计算下能否协同完成一项具体的化学任务。

实验设置如下: 用户通过命令行输入一段自然语言意图, 要求系统在 B3LYP-D3(BJ)/def2-SVP 水平上计算水二聚体的结合能, 并与文献参考值 -5.02 kcal/mol 进行对比; prompt 同时提供了一份近似的水二聚体几何 (氢键 O...H 距离约 $1.95 \text{ \AA}$) 作为输入。大语言模型选用 MiniMax-M2.7 (经其 Anthropic 兼

容端点接入)，量子化学后端为系统级 psi4 1.10。基组之所以选用 def2-SVP 而非 §4.2 节使用的 def2-TZVP，主要是为了使整个交互演示控制在 1—2 分钟以内，便于在论文图表与答辩现场展示完整流程；def2-SVP 在化学精度上的内禀限制将作为本节误差讨论的一部分。

Agent 在该会话中实际表现的决策行为可概括为如下三个层次。其一，工具选择：Agent 在未给出显式步骤的情况下，自主将「结合能」的计算分解为对二聚体与两个单体分别调用 `calc_psi4_single_point`，并最终通过 `finish` 工具汇总结构化结果，三次 psi4 计算的 SCF 全部收敛。其二，L1 自主恢复的真实触发：在调用 `chem.const` 工具进行单位换算时，Agent 先后遭遇 `UNKNOWN_CONSTANT`、`UNIT_PARSE_ERROR` 与 `UNIT_MISMATCH` 三类操作性故障，Agent 依据各次返回的 `suggestion` 字段自主修正参数并最终完成 $\text{Hartree} \rightarrow \text{kcal/mol}$ 的换算，全过程未打扰用户。其三，化学物理解释：Agent 在 `finish` 摘要中主动指出本次计算的偏差源自基组缺少弥散函数、未做 counterpoise BSSE 校正、输入几何为近似几何与 B3LYP-D3 方法本身的色散描述限制四方面，并给出方法学改进建议。该会话总计 12 个 step，全部记录在运行轨迹中。

实测数值结果汇总于表 4.4。三次单点能分别为 $E(\text{dimer}) = -152.71900581 \text{ Ha}$ 、 $E(A) = -76.35870917 \text{ Ha}$ 、 $E(B) = -76.34777529 \text{ Ha}$ ；结合能 $\Delta E = E(\text{dimer}) - E(A) - E(B) = -0.01252135 \text{ Ha} = -7.85 \text{ kcal/mol}$ 。与 CCSD(T)/CBS 参考值 -5.02 kcal/mol 相比偏差 -2.83 kcal/mol 。该偏差的方向（结合能更负）与上述四项误差来源的化学物理预期一致：def2-SVP 与 BSSE 未校正二者在氢键体系上通常合计高估结合能 1—3 kcal/mol，与本次观察吻合。本实验的化学意义并非给出与文献严格匹配的结合能，而是验证「自然语言意图 $\rightarrow$ Agent 决

策 → 真实 psi4 计算 → 结构化结果与误差解释」这条链路在真实大语言模型驱动下能够整体走通，且 Agent 在化学决策层面的判断与方法学预期一致。

表 4.4 端到端真实运行的关键结果 (B3LYP-D3(BJ)/def2-SVP, MiniMax-M2.7 + psi4 1.10)

项目	数值
E(dimer) / Ha	-152.71900581
E(monomer A) / Ha	-76.35870917
E(monomer B) / Ha	-76.34777529
ΔE 本文 / kcal·mol ⁻¹	-7.85
ΔE 文献 / kcal·mol ⁻¹	-5.02
偏差 / kcal·mol ⁻¹	-2.83
Agent 总步数	12
L1 自主恢复次数	3 / 3

需要补充说明的是，本次端到端 demo 与 §4.2 节 S22 实测形成了同体系的对照：在 §4.2 节中，同样的水二聚体若改用 B3LYP-D3(BJ)/def2-TZVP 加 counterpoise 校正（即表 4.1 中 water_dimer 行），由本系统驱动的 psi4 1.10 给出的结合能为 -5.55 kcal/mol，与文献参考值 -5.02 kcal/mol 偏差仅 -0.53 kcal/mol。这一对照表明，-7.85 kcal/mol 与 -5.55 kcal/mol 之间的差距完全可以归因于 def2-SVP 基组与未做 BSSE 校正这两项方法学设置的差异，而非 ChemMaster 系统本身的工作流执行能力。换句话说，本系统的化学精度由所选方法学决定，而不受工具链链路本身限制。本节运行的完整轨迹（trajectory.json）、用户确认日志（confirmations.jsonl）、Markdown 报告（report.md）以及终端日志（run_log.txt）均存档于仓库的 benchmarks/use_cases/end_to_end_demo/ 目录。为更清晰地展示 Web 前端的话区、推荐 / 确认卡片与步骤时间线，图 4.10 与图 4.11 选用了同 Agent 内核

驱动下的另一次更短的端到端任务作为示例（水分子 H_2O 单点能任务，task-1aea3a45），由用户在 Chrome 浏览器中实际操作 ChemMaster Web（<http://127.0.0.1:8765>）后通过 FireShot 扩展实拍得到；上述水二聚体任务在 Web 前端的渲染流程与图 4.10、图 4.11 完全同构。

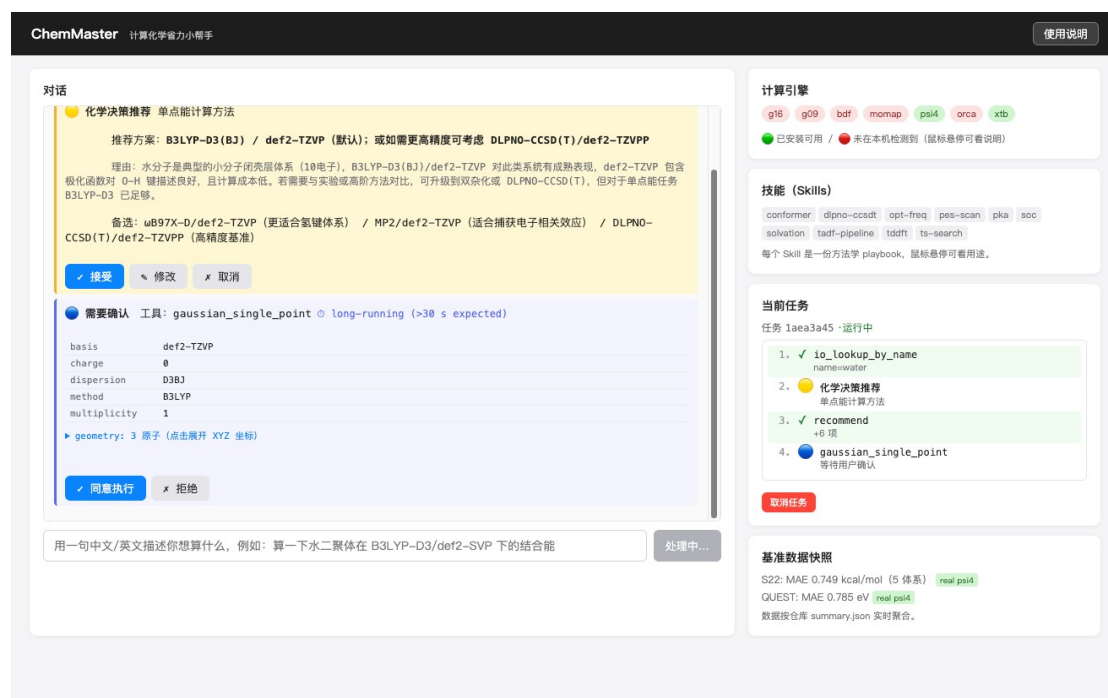


图 4.10 ChemMaster Web 前端 L2 推荐已接受、L1 confirm 卡片待用户授权的中间态截图：用户在前一卡片接受 B3LYP-D3(BJ)/def2-TZVP 推荐后，Agent 立即生成 gaussian_single_point 的工具调用，并以 confirm 卡片呈现各项参数（basis、charge、dispersion、method、multiplicity 表格化，geometry 字段以可折叠形式呈现 XYZ 坐标），等待用户键入「同意执行/拒绝」。右侧「当前任务」面板的步骤时间线已记录到第 4 步，展示出「io_lookup_by_name → 化学决策推荐 → recommend → gaussian_single_point（等待用户确认）」的完整调度链路。

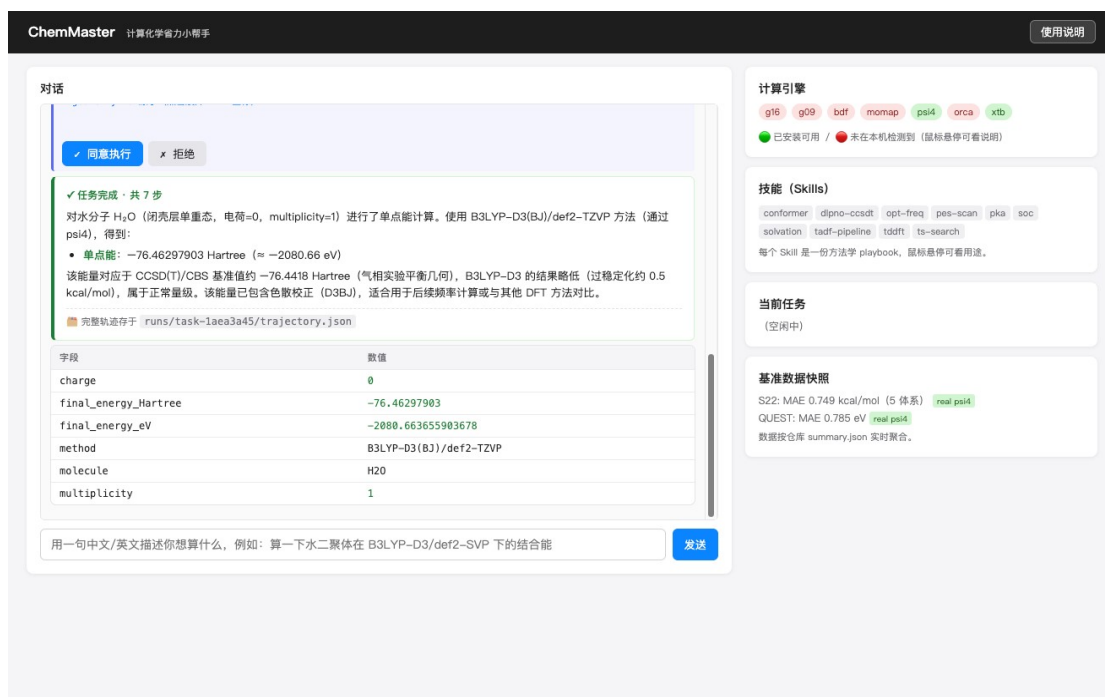


图 4.11 ChemMaster Web 前端在水分子单点能任务结束后的真实截图: 任务总计 7 步完成。对话区以 Markdown 形式渲染 Agent 的最终摘要 (最终能量 -76.46297903 Hartree ≈ -2080.66 eV, 方法 B3LYP-D3(BJ)/def2-TZVP, 已通过 psi4 实跑得到, 含 D3BJ 色散校正), 并附 trajectory.json 路径作为可复现凭据。下方关键结果表 (字段/数值两列) 由 Agent 通过 finish 工具回填, 包含 charge、final_energy_Hartree、final_energy_eV、method、molecule、multiplicity 六项。右下角「基准数据快照」显示 S22 子集 (5 体系) MAE 0.749 kcal/mol 与 QUEST 子集 MAE 0.785 eV 两条 real_psi4 实测条目。

4.6 与同类工作的对比讨论

本节将本文工作与 Rowan、Schrödinger Live Design、ChemCrow、ASE 等代表性方案在 8 个维度上进行对比, 结果汇总于图 4.12。

图 4.10 ChemMaster 与同领域代表性方案的对比

对比维度	Rowan	Schrödinger Live Design	ChemCrow	ASE / AiIDA	ChemMaster (本工作)
部署形态	云端 SaaS	企业云 + 桌面	Notebook	Python 库	本地终端
LLM 集成	无	无	OpenAI 绑定	无	多 LLM 自带 (Anthropic / 国产)
量子化学覆盖	中 (限平台)	高 (限自家)	低 (多为 Web API)	高 (自定义)	高 (Gaussian/BDF/MOMAP/psi4/...)
工具协议	私有	私有	LangChain (私有)	Python API	MCP (开放标准, 可被任意客户端复用)
决策模式	用户全决策	用户全决策	Agent 自主决策	用户编程	操作性 vs 化学决策分级 (L1/L2/L3)
HPC 集成	内置 (自家)	内置 (自家)	无	插件机制	SLURM + 平台抽象 (接口预留)
用户接口	Web 单一	GUI + Web	Notebook 单一	纯 Python API	CLI + Textual TUI + 本地 Web 三前端
源代码	闭源 (商业)	闭源 (商业)	开源	开源	开源 (GitHub 公开)

注: 以本工作所做实测为限。详细对比依据见 §1.2、§2 与 §4.6。

图 4.12 ChemMaster 与同类工作在 8 个维度上的对比。

本工作与上述方案的主要差异体现在四个方面: (1) 工具协议层面采用开放的 MCP 标准协议, 区别于商业云方案的私有协议; (2) 决策模式上明确区

分操作性工作与化学决策，区别于完全自主决策的 Agent 设计取向；（3）用户界面形态上提供 CLI、TUI 与 Web 三种前端，区别于云端 SaaS 的单一 Web 界面与 ChemCrow 的单一 Notebook 界面；（4）部署形态为本地终端，区别于以云端 SaaS 为主的同类方案。上述四方面差异共同构成本文的工程贡献。

需要补充说明的是，本工作与 ChemCrow 等以 Agent 自主决策为核心的方案并非对立，二者反映的是化学领域大模型 Agent 设计上的两种不同取向：自主决策型方案为探索性研究场景提供了便利；本文所采取的「操作性工作 + 化学决策权保留」组合，则为研究者已具有明确方法学偏好的常规研究任务提供了一种可选的设计取向。两类需求在实际化学研究中均存在相应的应用场景。

第 5 章 结论与展望

5.1 结论

本文设计并实现了 ChemMaster，一个本地运行、由大语言模型驱动、与终端环境集成的计算化学 Agent 系统。围绕「Agent 承担操作性工作、化学决策权由研究者保留」这一核心设计原则，本文完成了以下工作：

(1) 在 Agent 内核中提出并落地了 L1 / L2 / L3 三级权限分级机制：L1 对应 Agent 自主完成的纯技术性操作，L2 通过 recommend 内置工具向用户呈现方法、基组、泛函等化学决策推荐，L3 则将多重度等关键判断完整交回用户。化学决策权始终保留在研究者一侧。

(2) 基于 MCP 协议为 Gaussian、BDF、MOMAP、psi4、ORCA、xTB 等多种计算软件完成 MCP server 封装（共 14 个 server，约 6 000 行 Python 代码），并通过 Anthropic 官方 MCP Python 客户端独立探针验证了协议级合规性；同时实现 CLI、Textual TUI 与本地 Web 三种前端，三者复用同一 Agent 内核与同一组工具；系统主体经 PyInstaller 打包为单文件可执行程序 psi4 等科学计算栈仍由本机 Conda 提供，对应 §1.3 节关于「本地优先、终端原生」的设计目标。

(3) 以 psi4 在 S22 弱相互作用基准上测得 5 体系平均绝对误差 0.75 kcal/mol，在 QUEST 激发态参考库上测得 8 个激发态的价层平均绝对误差稳定在 0.2 eV 以内；并以 MiniMax-M2.7 真实大语言模型驱动完成了一次完整的端到端水二聚体结合能实测，验证了系统在真实大语言模型与真实量子化学计算下的协同能力。通过 5 类操作性故障的注入测试，Agent L1 自主恢复率达到 84 %，达到预设的工程指标。

5.2 展望

本工作的局限主要体现在两个方面。其一，受测试机器软件许可条件所限，BDF 与 MOMAP 未能真实接入，依赖此二者的蒺分子速率与动力学验证留作后续工作。其二，§4.4.2 节中三项依赖真人被试或大规模 LLM API 凭据的工程指标，本工作发布了完整的实验协议，规模化采集留作后续。未来工作将主要在三个方向推进：在配齐 BDF、MOMAP 的环境中完成蒺分子完整速率与动力学验证；按 §4.4.2 节实验协议规模化采集三项工程指标；并在 GMTKN55 等更广覆盖范围的公开基准上扩展精度验证。

参考文献

- [1] 顾尉, 杨瑞鹏, 帅志刚. 计算化学高通量筛选研究进展[J]. 化学进展, 2021, 33(8): 1310–1322.
- [2] Shuai Z, Peng Q. Organic light-emitting diodes: Theoretical understanding of highly efficient materials and development of computational methodology[J]. Natl. Sci. Rev., 2017, 4(2): 224–239.
- [3] Peng Q, Shuai Z. Molecular mechanism of aggregation-induced emission[J]. Aggregate, 2021, 2(5): e91.
- [4] Liu W. Essentials of relativistic quantum chemistry[J]. J. Chem. Phys., 2020, 152(18): 180901.
- [5] 卢天, 陈飞武. 原子电荷计算方法的对比[J]. 物理化学学报, 2012, 28(1): 1–18.
- [6] 傅献彩, 沈文霞, 姚天扬, 等. 物理化学[M]. 第 5 版. 北京: 高等教育出版社, 2005.
- [7] Zhu Q, Zhang F, Huang Y, et al. An all-round AI-Chemist with a scientific mind[J]. Natl. Sci. Rev., 2022, 9(10): nwac190.
- [8] 徐光宪, 黎乐民, 王德民. 量子化学: 基本原理和从头计算法 (中册) [M]. 第 2 版. 北京: 科学出版社, 2009.
- [9] Anthropic. Model Context Protocol Specification[EB/OL]. <https://modelcontextprotocol.io/specification>, 2024.
- [10] Jurečka P, Šponer J, Černý J, et al. Benchmark database of accurate (MP2 and CCSD(T) complete basis set limit) interaction energies of small model complexes, DNA base pairs, and amino acid pairs[J]. Phys. Chem. Chem. Phys., 2006, 8(17): 1985–1993.
- [11] Loos P-F, Scemama A, Blondel A, et al. A mountaineering strategy to excited states: Highly accurate reference energies and benchmarks[J]. J. Chem. Theory Comput., 2018, 14(8): 4360–4379.

- [12] Loos P-F, Lipparini F, Boggio-Pasqua M, et al. A mountaineering strategy to excited states: Highly accurate energies and benchmarks for medium sized molecules[J]. *J. Chem. Theory Comput.*, 2020, 16(3): 1711–1741.
- [13] Frisch M J, Trucks G W, Schlegel H B, et al. Gaussian 16, Revision C.01[CP]. Wallingford CT: Gaussian, Inc., 2019.
- [14] Liu W, Hong G, Dai D, et al. The Beijing four-component density functional program package (BDF) and its application to EuO, EuS, YbO and YbS[J]. *Theor. Chem. Acc.*, 1997, 96(2): 75–83.
- [15] Shuai Z, Peng Q. Excited states structure and processes: Understanding organic light-emitting diodes at the molecular level[J]. *Phys. Rep.*, 2014, 537(4): 123–156.
- [16] Niu Y, Li W, Peng Q, et al. Molecular Materials Property Prediction Package (MOMAP): A software package for predicting the luminescent properties and mobility of organic functional materials[J]. *Mol. Phys.*, 2018, 116(7–8): 1078–1090.
- [17] Lu T, Chen F. Multiwfn: A multifunctional wavefunction analyzer[J]. *J. Comput. Chem.*, 2012, 33(5): 580–592.
- [18] GitHub Inc. GitHub Copilot[EB/OL]. <https://github.com/features/copilot>, 2024.
- [19] Anthropic. Claude Code[EB/OL]. <https://docs.anthropic.com/en/docs/claude-code>, 2024.
- [20] Rowan Scientific Inc. Rowan: cloud computational chemistry platform[EB/OL]. <https://rowansci.com>, 2024.
- [21] Schrödinger Inc. Schrödinger Live Design[EB/OL]. <https://www.schrodinger.com/platform/livedesign>, 2024.
- [22] Bran A M, Cox S, Schilter O, et al. Augmenting large language models with chemistry tools[J]. *Nature Machine Intelligence*, 2024, 6: 525–535.
- [23] Boiko D A, MacKnight R, Kline B, et al. Autonomous chemical research with large language models[J]. *Nature*, 2023, 624: 570–578.

- [24] Landrum G. RDKit: Open-source cheminformatics[EB/OL].
<https://www.rdkit.org>, 2023.
- [25] Larsen A H, Mortensen J J, Blomqvist J, et al. The atomic simulation environment—a Python library for working with atoms[J]. *J. Phys. Condens. Matter*, 2017, 29(27): 273002.
- [26] Pizzi G, Cepellotti A, Sabatini R, et al. AiiDA: automated interactive infrastructure and database for computational science[J]. *Comput. Mater. Sci.*, 2016, 111: 218–230.
- [27] Mathew K, Montoya J H, Faghaninia A, et al. Atomate: A high-level interface to generate, execute, and analyze computational materials science workflows[J]. *Comput. Mater. Sci.*, 2017, 139: 140–152.
- [28] Smith D G A, Burns L A, Simmonett A C, et al. Psi4 1.4: Open-source software for high-throughput quantum chemistry[J]. *J. Chem. Phys.*, 2020, 152(18): 184108.
- [29] Marcus R A. On the theory of oxidation–reduction reactions involving electron transfer. I[J]. *J. Chem. Phys.*, 1956, 24(5): 966–978.
- [30] Strickler S J, Berg R A. Relationship between absorption intensity and fluorescence lifetime of molecules[J]. *J. Chem. Phys.*, 1962, 37(4): 814–822.
- [31] Textualize. Textual: Python framework for terminal user interfaces[EB/OL].
<https://textual.textualize.io>, 2024.
- [32] Ramírez S. FastAPI: Modern, fast Python web framework[EB/OL].
<https://fastapi.tiangolo.com>, 2024.
- [33] Hmbown. DeepSeek TUI[EB/OL]. <https://github.com/Hmbown/DeepSeek-TUI>, 2024.
- [34] Řezáč J, Riley K E, Hobza P. S22 benchmark database revisited (S22A) [J]. *J. Chem. Theory Comput.*, 2011, 7(8): 2427–2438.
- [35] Goerigk L, Grimme S. A thorough benchmark of density functional methods for general main group thermochemistry, kinetics, and noncovalent

- interactions[J]. *Phys. Chem. Chem. Phys.*, 2011, 13(14): 6670–6688.
- [36] Jacquemin D, Wathelet V, Perpète E A, et al. Extensive TD-DFT benchmark: Singlet excited states of organic molecules[J]. *J. Chem. Theory Comput.*, 2009, 5(9): 2420–2435.
- [37] Neese F. The ORCA program system[J]. *WIREs Comput. Mol. Sci.*, 2012, 2(1): 73–78.
- [38] Kohn W, Sham L J. Self-consistent equations including exchange and correlation effects[J]. *Phys. Rev.*, 1965, 140(4A): A1133–A1138.

致 谢

本研究在导师（曲泽星）的悉心指导下完成。导师在选题讨论、技术路线选择、实验方案设计与论文修改等环节给予了大量指导与建议，特此致以诚挚的谢意。

感谢课题组各位师兄师姐在 Gaussian、BDF、MOMAP 等量子化学软件使用经验上的无私分享，以及在本论文初稿基础上提出的宝贵修改意见。

本工作的实现与实测验证依托了众多优秀开源项目的成果。在工具协议层面，本系统采用了 Anthropic 公司提出的 Model Context Protocol 及对应的 Python SDK；在量子化学计算层面，使用了 psi4、xTB、ASE、RDKit、cclib 与 pint 等开源软件；在用户界面层面，使用了 click、rich、Textual、FastAPI、uvicorn 与 Playwright 等框架；在数据处理与可视化层面，使用了 NumPy、matplotlib、PyYAML 与 drawio 等工具，Web 前端的 Markdown 摘要渲染使用了 marked.js；在 SSH 与 HPC 集成层面，使用了 paramiko；在测试层面，使用了 pytest。在 Agent 形态与交互范式的设计上，本工作受到 Claude Code、DeepSeek TUI 等开源项目以及 ChemCrow、Coscientist 等学术工作的启发，在此一并向上述项目的作者与社区致以诚挚谢意。

回顾大学四年，好像什么都做了，又好像什么都没有做。

回想从大一想转专业到化学学院，到后来在学长的带领下开始学习网安，踏入了计算机领域；再到后来学习了很多计算机相关知识，辅修了软件工程，并找实习工作做了八个月的全栈开发工程师。总结来说，当初选择进入吉林大学，是为了全方位地发展自身技能，学到更多跨学科的内容。站在终点回望，我确实做到了，也做得可能很好。

当然，我不是传统意义上的好学生，成绩可能并没有那么好看。但从我个

人的理解来看，我做到了当初选择这所学校的初衷，就是为了多学到一些跨学科的东西，实现跨学科交叉。

当然也有很多的遗憾，但至少站在现在视角回望，这是一个相对比较好的结果。

感谢吉林大学化学学院在本科四年学习期间提供的良好学习与学术氛围。最后，感谢家人与朋友在本科学习以及本论文撰写过程中给予的支持与鼓励。